

Bayesian Knowledge Tracing (BKT)

Comment la plateforme estime ce que chaque étudiant maîtrise — expliqué depuis ton code

Plateforme d'apprentissage adaptatif · Yassine Ben Nessib · EspritTech — expliqué à partir du vrai code source

En une phrase — Le BKT garde une probabilité « quelle est la chance que cet étudiant maîtrise ce concept ? » et la met à jour après chaque réponse avec la règle de Bayes, en corrigeant les bonnes réponses par chance et les fautes d'inattention. Tout est dans `bkt_service.py`.

1 L'idée en une minute

On ne voit que les réponses aux quiz, et elles sont **bruitées** : un étudiant peut avoir raison par **chance** (un “guess”) ou tort par **inattention** (un “slip”). Une simple moyenne se fait piéger. Le BKT garde plutôt une **probabilité** de maîtrise et la met à jour soigneusement après chaque réponse. Il remplace l'ancienne règle `mastery = 0.6*old + 0.4*new`.

2 Où ça vit dans le code

`app/services/bkt_service.py`

Le modèle lui-même : les paramètres + la mise à jour de Bayes.

`app/data/bkt_params.json`

Paramètres ajustés par concept (sinon les valeurs par défaut).

`scripts/fit_bkt.py`

Apprend les 4 paramètres depuis de vrais logs (algorithme EM).

Modèle Mastery (BD)

Stocke la probabilité par (étudiant, concept).

3 Les quatre nombres utilisés par le modèle

BKT_SERVICE.PY — LES PARAMÈTRES PAR DÉFAUT

```
DEFAULT_PARAMS = {  
    "p_L0": 0.20,    # prior knowledge (before any practice)  
    "p_T": 0.15,     # learning rate (chance to learn between two questions)  
    "p_S": 0.10,     # slip (knows, but answers wrong)  
    "p_G": 0.25,     # guess (does not know, but answers right -> 1/4 on a 4-choice MCQ)  
}
```

Ce que ça fait, étape par étape :

- Un `dict` Python (une “boîte” de valeurs étiquetées) contient les quatre paramètres du BKT.
- **p_L0** = à quel point l’étudiant connaît déjà le concept au tout début.
- **p_T** = la probabilité qu’il l’apprenne entre deux questions.
- **p_S** (slip) et **p_G** (guess) modélisent le bruit. Guess vaut 0.25 car une question à 4 choix est juste 1 fois sur 4 par pur hasard.
- `get_params(concept_id)` renvoie ces valeurs par défaut, OU les valeurs ajustées de `bkt_params.json` si elles existent pour ce concept.

4 Le cœur : une étape de mise à jour (Bayes + apprentissage)

C’est la fonction la plus importante. Chaque réponse déclenche **deux étapes** : (1) corriger notre croyance grâce à la réponse, puis (2) tenir compte du fait que l’étudiant vient peut-être d’apprendre.

BKT_SERVICE.PY — POSTERIOR_AFTER_OBS() (LE CŒUR)

```
def posterior_after_obs(p_known, correct, params):
    s, g = params["p_S"], params["p_G"]
    p = _clamp(p_known)          # keep p strictly between 0 and 1
    if correct:                  # Bayes update if the answer is CORRECT
        num = p * (1.0 - s)
        den = p * (1.0 - s) + (1.0 - p) * g
    else:                        # Bayes update if the answer is WRONG
        num = p * s
        den = p * s + (1.0 - p) * (1.0 - g)
    p_post = (num / den) if den > 1e-9 else p
    # Learning transition: maybe the student just learnt it.
    p_next = p_post + (1.0 - p_post) * params["p_T"]
    return _clamp(p_next)
```

Ce que ça fait, étape par étape :

- **Entrées :** `p_known` = notre croyance actuelle (un nombre 0-1), `correct` = Vrai/Faux, et les 4 paramètres.
- `_clamp(p)` force simplement `p` à rester entre 0 et 1 (jamais exactement 0 ni 1), pour éviter une division par zéro.
- **Si la réponse est correcte :** on applique Bayes. `num` = la chance d'avoir juste *parce qu'on sait* ($p \times \text{non-slip}$). `den` = la chance totale d'avoir juste (savoir & pas de slip + ne pas savoir & deviner). La fraction `num/den` est la nouvelle croyance.
- **Si elle est fausse :** même idée à l'envers — avoir tort en sachant (slip) vs avoir tort sans savoir.
- **Les deux dernières lignes** = l'étape d'apprentissage : même après la réponse, l'étudiant a pu apprendre, donc on remonte la croyance de `p_T`.
- Elle renvoie `p_next` = la probabilité de maîtrise mise à jour. C'est ce nombre que la plateforme enregistre.

LA MÊME CHOSE EN FORMULE (RÉPONSE CORRECTE)

$$P(\text{connu} \mid \text{correct}) = [p \cdot (1 - s)] / [p \cdot (1 - s) + (1 - p) \cdot g]$$
 puis
$$P(\text{suivant}) = P(\text{connu} \mid \text{correct}) + (1 - P(\text{connu} \mid \text{correct})) \cdot p_T$$

5 Le faire tourner sur tout un quiz

BKT_SERVICE.PY — UPDATE_MASTERY() (BOUCLE SUR LES RÉPONSES)

```
def update_mastery(concept_id, prior_p, observations):
    params = get_params(concept_id)
    p = params["p_L0"] if prior_p is None else _clamp(prior_p)
    for correct in observations:          # one update per answer
        p = posterior_after_obs(p, bool(correct), params)
    return p
```

Ce que ça fait, étape par étape :

- `observations` est une liste de Vrai/Faux (les réponses de l'étudiant sur ce concept).
- On part de la croyance enregistrée `prior_p`, ou de `p_L0` si l'étudiant est nouveau.
- La boucle `for` applique la mise à jour une fois par réponse, en réinjectant le résultat — la croyance évolue pas à pas.
- Elle renvoie la probabilité finale de maîtrise après tout le quiz.

6 Prédire la prochaine réponse

BKT_SERVICE.PY — PREDICT_CORRECT()

```
def predict_correct(concept_id, p_known):
    params = get_params(concept_id)
    p = _clamp(p_known)
    return p * (1.0 - params["p_S"]) + (1.0 - p) * params["p_G"]
```

Ce que ça fait, étape par étape :

- À partir de la maîtrise actuelle `p_known`, on estime la chance que l'étudiant réponde correctement à la question suivante.
- On mélange deux cas : savoir & ne pas faire de slip, OU ne pas savoir & deviner juste. Le moteur adaptatif s'en sert pour choisir une question ni trop facile ni trop dure.

7 Comment circulent les données à l'exécution

- 1 **L'étudiant répond à un quiz** — chaque question est correcte/incorrecte sur un concept.
- 2 **update_mastery() s'exécute** — en partant de la probabilité enregistrée, on applique une étape Bayes+apprentissage par réponse.
- 3 **La maîtrise est sauvegardée** — la nouvelle probabilité est stockée par (étudiant, concept) en base.
- 4 **Le moteur adaptatif réagit** — maîtrise faible → révision / contenu plus simple ; maîtrise haute → on avance.

8 Exemple chiffré (vrais nombres)

Une réponse correcte

Départ $p=0.30$, avec $G=0.25$, $S=0.10$, $T=0.15$.

Correct $\rightarrow \text{num} = 0.30 \times 0.90 = 0.27$; $\text{den} = 0.27 + 0.70 \times 0.25 = 0.445 \rightarrow p_{\text{post}} = 0.27/0.445 \approx \mathbf{0.607}$.

Apprentissage $\rightarrow p_{\text{next}} = 0.607 + (1-0.607) \times 0.15 \approx \mathbf{0.666}$.

Une seule bonne réponse fait passer la croyance de 30 % à ~67 % — pas 100 %, car ça pourrait être un guess.

9 Forces & limites

Forces

- Probabilité claire et interprétable
- Seulement 4 paramètres — peu de données nécessaires
- Gère correctement guess & slip
- Un standard éprouvé en EdTech

Limites

- Suppose qu'on n'oublie jamais
- Connaissance binaire (sait / ne sait pas)
- Concepts traités indépendamment
- Les paramètres doivent être fixés ou ajustés

Note honnête — Les 4 paramètres utilisent des valeurs par défaut raisonnables et peuvent être **ajustés** sur de vrais logs avec `scripts/fit_bkt.py` (un algorithme EM). En attendant assez de vraies données, les valeurs par défaut sont utilisées — dis-le honnêtement si on te le demande.

10 Q&R

QUESTIONS DE SOUTENANCE / ENTRETIEN (DONT « CE BLOC FAIT QUOI ? »)

Q Que fait la fonction `posterior_after_obs` ?

R Elle effectue une étape de BKT sur une seule réponse : une mise à jour de Bayes (corrigeant slip/guess) suivie de la transition d'apprentissage. Elle renvoie la nouvelle probabilité de maîtrise.

Q Pourquoi deux branches (si correct / sinon) ?

R Parce que la règle de Bayes diffère pour une bonne et une mauvaise réponse. Une bonne réponse augmente la croyance (sauf si c'était un guess) ; une mauvaise la baisse (sauf si c'était un slip).

Q À quoi sert `_clamp` et pourquoi ?

R Un petit utilitaire qui garde la probabilité strictement entre 0 et 1, pour ne jamais diviser par zéro ni rester bloqué sur une valeur qui ne peut plus changer.

Q Pourquoi p_G vaut 0.25 ?

R Nos quiz sont à 4 choix, donc deviner au hasard est juste environ 1 fois sur 4.

Q Comment améliorer le BKT ?

R Ajuster les paramètres par concept sur de vrais logs (`fit_bkt.py`), ajouter l'oubli, ou passer au Deep Knowledge Tracing (un modèle neuronal) quand il y aura assez de données.

11 Termes clés

dict — Une boîte étiquetée Python : nom $\rightarrow$ valeur (ici les 4 paramètres).

Prior — Croyance avant la nouvelle réponse (p_{L0} ou la valeur enregistrée).

Posterior — Croyance après la réponse (le résultat de Bayes).

Slip / Guess — Le bruit : se tromper en sachant / réussir sans savoir.

Transition (p_T) — La chance d'avoir appris entre deux questions.

Clamp — Forcer un nombre à rester dans un intervalle.